

Supply Chain and Warehouse Logistics Network

Phase 1 Conceptual Design

This report presents the conceptual design of a database that tracks inventory across warehouses in different geographical locations, supplier orders, shipments, and delivery events. The model addresses two central requirements: multi-level product structures and restock alerts when inventory falls below its reorder level.

The Phase 1 result defines the system scope, 11 functional requirements, 15 business rules, 13 entities, and 17 relationships with explicit cardinality and participation. It also provides a complete EER model and conceptual logic for recursive product components and inventory alerts.

Information	Details
Course	Database Systems
Topic	6 Supply Chain and Warehouse Logistics Network
Stage	Phase 1 Conceptual Design
Submission	Progress report and EER model

Completed Work

- Defined the problem, users, scope, and assumptions of the system.
- Modelled warehouses, products, suppliers, supplier orders, shipments, tracking events, and inventory.
- Represented multi-level product structures through the recursive ProductComponent entity.
- Specified when a restock alert must be created, retained, closed, and opened again.
- Checked completeness with acceptance cases at the end of the report.

I. Problem Context and Scope

1.1 Problem Statement

The organization operates warehouses in different cities or countries. A product may be stored in several warehouses, a supplier may provide several products, and one supplier

order may be delivered through several shipments. Fragmented records make it difficult to confirm the stock held at each location, compare ordered and delivered quantities, track shipments, and identify products that require replenishment.

Some finished products are assembled from components that may contain their own subcomponents. A flat product list cannot represent this structure or support material requirement calculations. The database must support an unlimited number of component levels while preventing a product from containing itself directly or indirectly.

1.2 Objectives

- Manage warehouses, products, suppliers, and supplier product details in one system.
- Maintain one inventory balance for every warehouse and product combination.
- Track supplier orders, order items, partial shipments, and shipment events over time.
- Represent multi-level product structures with the quantity required at each level.
- Record every stock movement and issue an alert when inventory is below its reorder level.
- Provide a clear EER model that can be mapped to a relational design in Phase 2.

1.3 Users and Main Workflow

User Role	Main Responsibility
Purchasing staff	Manage suppliers, supplier prices, and supplier orders.
Warehouse staff	Confirm receipts, issues, adjustments, and current quantities.
Logistics staff	Create shipments and update shipment tracking events.
Inventory manager	Review reorder levels, low-stock alerts, and replenishment decisions.

A typical workflow starts when purchasing staff create a supplier order. Logistics staff then create one or more shipments and record tracking events. When warehouse staff confirm receipt, the system records a stock movement, updates the inventory quantity, and evaluates the restock condition within the same transaction.

1.4 Scope and Assumptions

Category	Details
In scope	Geographical warehouses; products and suppliers; current supplier prices; supplier orders; partial shipments; tracking events; inventory; stock movements; multi-level product structures; restock alerts.
Outside Phase 1	Customer sales; route optimization; production batches; serial numbers; returns; warehouse transfers; unit conversion; BOM version history; payments and accounting.
Order assumptions	Each order has one supplier and one destination warehouse. A product appears at most once in the same order.

Category	Details
Data assumptions	Each product uses one base unit; prices use one common currency; event times use the same standard time zone.

II. Functional Requirements

ID	Function	Requirement
FR01	Warehouse management	Store the code, name, address, city, country, and capacity of every warehouse.
FR02	Product management	Store product details, base unit, and product type.
FR03	Supplier management	Store contact details and the products currently offered by each supplier.
FR04	Supplier orders	Create an order for one supplier, one destination warehouse, and several product lines.
FR05	Partial deliveries	Allocate one order item to several shipments without exceeding the ordered quantity.
FR06	Shipment tracking	Store a tracking number, dispatch date, expected date, arrival date, status, and chronological events.
FR07	Inventory management	Store the current quantity and reorder level for each warehouse and product pair.
FR08	Stock movements	Create one StockMovement whenever a balance is opened, received, issued, or adjusted.
FR09	Product structure	Store the parent product, component product, and required quantity at every level.
FR10	BOM cycle prevention	Prevent a product from containing itself directly or indirectly.
FR11	Restock alerts	Create one open alert when Quantity is below ReorderLevel and close it after stock recovers.

III Business Rules

ID	Business Constraint
BR01	Each Inventory record belongs to exactly one Warehouse and one Product. A warehouse or product may have zero or many inventory records. WarehouseID and ProductID must be unique as a pair.
BR02	Supplier and Product have a many-to-many relationship through SupplierProduct. Each supplier product record stores the current UnitPrice and LeadTimeDays.
BR03	Each SupplierOrder refers to exactly one Supplier and one destination Warehouse. A supplier or warehouse may have zero or many orders.
BR04	A draft order may have no items, but a submitted order must have at least one. Each SupplierOrderItem belongs to exactly one order and one product, and a product cannot repeat in the same order.
BR05	The order supplier must have a SupplierProduct record for every ordered product. UnitPrice on SupplierOrderItem is the agreed historical price and does not change with the current supplier price.
BR06	An order may have zero or many Shipments. Each Shipment belongs to exactly one order and inherits the destination warehouse from that order.
BR07	A draft shipment may have no items, but a dispatched shipment must have at least one ShipmentItem. Each ShipmentItem refers to a SupplierOrderItem from the same order as the shipment.
BR08	A SupplierOrderItem may be divided among several shipments. The total quantity of non-cancelled ShipmentItems must not exceed the ordered quantity.
BR09	Each TrackingEvent belongs to exactly one Shipment. A shipment may have zero or many events. TrackingEventID distinguishes events that occur at the same time.
BR10	Product contains Product is a recursive many-to-many relationship. ProductComponent stores the parent product, component product, and a QuantityRequired value greater than zero.
BR11	A product must not contain itself directly or indirectly. A product without components is a leaf item, while a component may still contain lower-level components.
BR12	Inventory.Quantity and ReorderLevel cannot be negative. A stock issue is rejected if it would make Quantity negative. Each successful change creates exactly one StockMovement.
BR13	A Receipt movement refers to exactly one ShipmentItem, and each ShipmentItem may be received at most once. Other movement types do not refer to a shipment item.
BR14	Low stock occurs when Quantity is less than ReorderLevel. Each Inventory record may have at most one unresolved RestockAlert. Repeated low-stock updates must not create duplicate alerts.
BR15	When Quantity is greater than or equal to ReorderLevel, the open alert is resolved. If Quantity later falls below the threshold, a new alert period begins. A change to ReorderLevel must also be evaluated.

IV. Entities, Attributes, Relationships and Definitions

This section focuses on the definitions required for the Phase 1 conceptual model. Each entity is explained in normal prose, while the attribute, relationship, and function details are kept in tables for easier reading and checking.

4.1 System Definition

The Supply Chain and Warehouse Logistics Network Database manages products across multiple warehouses, suppliers, supplier orders, shipments, inventory transactions, multi-level product structures, and restock alerts. It provides a consistent view of what products exist, where they are stored, which suppliers offer them, what has been ordered, how orders are delivered, how inventory changes, and when stock falls below its reorder level.

4.2 User / Actor Definitions

4.2.1 Purchasing Staff: Manage supplier information, supplier-product offerings and current prices, create and manage supplier orders, and monitor order status.

4.2.2 Warehouse Staff: Handle warehouse-side inventory operations, including viewing inventory, confirming shipment receipts, and performing issue/adjustment activities through the stock-movement process.

4.2.3 Logistics Staff: Create shipments, allocate order quantities to shipments, dispatch shipments, confirm arrival, and record tracking events.

4.2.4 Inventory Manager: Review inventory levels, reorder conditions, low-stock alerts, and replenishment decisions.

4.3 Entity Definitions

4.3.1 Warehouse

Definition: Represents a physical storage location. It stores warehouse identification and location/capacity information.

Entity type: Strong (independent) entity.

Identifier: WarehouseID

Relationship meaning: A Warehouse may have 0..N Inventory records and may be the destination of 0..N SupplierOrders.

4.3.2 Product

Definition: Represents a stocked, purchased, or assembled item.

Entity type: Strong (independent) entity.

Identifier: ProductID

Relationship meaning: A Product may have 0..N Inventory and SupplierProduct records, may appear in many SupplierOrderItems, and may participate as parent or component in ProductComponent.

4.3.3 Inventory

Definition: Represents the current stock balance of one Product at one Warehouse.

Entity type: Associative entity depending on Warehouse and Product.

Identifier: InventoryID

Relationship meaning: Each record belongs to exactly one Warehouse and one Product; the WarehouseID + ProductID pair is unique.

4.3.4 Supplier

Definition: Represents a supplier/product source.

Entity type: Strong (independent) entity.

Identifier: SupplierID

Relationship meaning: A Supplier may offer many Products through SupplierProduct and may be referenced by many SupplierOrders.

4.3.5 SupplierProduct

Definition: Represents a supplier's offering of a product, including current price and lead time.

Entity type: Associative entity (Supplier $\times$ Product).

Identifier: SupplierID + ProductID

Relationship meaning: Resolves the many-to-many Supplier–Product relationship.

4.3.6 SupplierOrder

Definition: Represents a purchase order placed with a supplier.

Entity type: Strong entity (transaction header).

Identifier: OrderID

Relationship meaning: Each order has exactly one Supplier and one destination Warehouse; a submitted order must contain at least one item.

4.3.7 SupplierOrderItem

Definition: Represents one product line within a supplier order.

Entity type: Weak entity depending on SupplierOrder.

Identifier: OrderItemID

Relationship meaning: Each item belongs to exactly one order and one Product; a Product cannot repeat in the same order; UnitPrice is the historical agreed price.

4.3.8 Shipment

Definition: Represents a physical delivery made against a supplier order.

Entity type: Strong entity (transaction header).

Identifier: ShipmentID

Relationship meaning: A SupplierOrder may have 0..N Shipments; each Shipment belongs to one order and inherits its destination Warehouse.

4.3.9 ShipmentItem

Definition: Represents the quantity of an order line allocated to a shipment.

Entity type: Weak/associative entity depending on Shipment and SupplierOrderItem.

Identifier: ShipmentItemID

Relationship meaning: Supports partial delivery; total non-cancelled allocations cannot exceed the ordered quantity.

4.3.10 TrackingEvent

Definition: Represents a shipment tracking/status event.

Entity type: Weak entity depending on Shipment.

Identifier: TrackingEventID

Relationship meaning: Each event belongs to one Shipment; a Shipment may have 0..N events.

4.3.11 ProductComponent

Definition: Represents a direct parent–component relationship in a multi-level product structure.

Entity type: Recursive associative entity (Product $\times$ Product).

Identifier: ParentProductID + ComponentProductID

Relationship meaning: The same Product entity appears in parent and component roles; direct or indirect cycles are not allowed.

4.3.12 StockMovement

Definition: Represents a successful inventory transaction and preserves stock-change history.

Entity type: Weak entity depending on Inventory; optionally ShipmentItem.

Identifier: MovementID

Relationship meaning: Receipt movements may reference a ShipmentItem; other movement types do not.

4.3.13 RestockAlert

Definition: Represents one low-stock alert period.

Entity type: Weak entity depending on Inventory.

Identifier: AlertID

Relationship meaning: An Inventory record may have at most one unresolved alert; a later low-stock period creates a new alert after the previous one is resolved.

4.4 Attribute Definitions and Data Dictionary

4.4.1 Warehouse

WarehouseID is the unique identifier of a warehouse. It distinguishes one warehouse from another and is used as the primary key of the Warehouse entity. Its data type is **INT** and it must be required and unique.

WarehouseName stores the name of the warehouse. Its data type is **VARCHAR(100)**. The value is required because every warehouse must have a name.

Address stores the physical address of the warehouse. Its data type is **VARCHAR(200)**. The address is required to identify the warehouse's physical location.

City stores the city in which the warehouse is located. Its data type is **VARCHAR(100)** and the value is required.

Country stores the country in which the warehouse is located. Its data type is **VARCHAR(100)** and the value is required.

CapacityM3 stores the storage capacity of the warehouse in cubic meters. Its data type is **DECIMAL(12,2)**. The value represents the maximum storage capacity and must not be negative.

4.4.2 Product

ProductID is the unique identifier of a product. It is the primary key of the Product entity. Its data type is **INT** and it must be required and unique.

ProductName stores the name used to identify the product. Its data type is **VARCHAR(150)** and the value is required.

Description stores additional information describing the product. Its data type is **TEXT**. This attribute provides descriptive information that is not captured by the other product attributes.

Unit stores the base unit used to measure the product quantity, such as a unit of measure used consistently throughout the system. Its data type is **VARCHAR(20)**. Each product uses one base unit according to the Phase 1 data assumptions.

ProductType identifies the type or category of the product. Its data type is **VARCHAR(50)**. The exact controlled list of product types is not fixed in Phase 1 and can be finalized during Phase 2.

4.4.3 Inventory

InventoryID is the unique identifier of an inventory record. Its data type is **INT** and it is the primary key. It must be required and unique.

WarehouseID identifies the warehouse where the inventory is stored. It is a foreign key referencing **Warehouse** and is required. Each Inventory record must belong to exactly one warehouse.

ProductID identifies the product represented by the inventory record. It is a foreign key referencing **Product** and is required. Each Inventory record must belong to exactly one product.

Quantity stores the current quantity of the product available at the specified warehouse. Its data type is **DECIMAL(12,2)**. The value cannot be negative. A stock issue must be rejected if it would cause Quantity to become negative.

ReorderLevel stores the minimum inventory threshold used to determine whether replenishment is required. Its data type is **DECIMAL(12,2)**. The value cannot be

negative. When Quantity becomes lower than ReorderLevel, the system identifies a low-stock condition.

LastUpdated stores the date and time when the current inventory balance was most recently changed. Its data type is **DATETIME** and it is used to maintain the currency of the inventory information.

The combination of **WarehouseID** and **ProductID** must be unique, because the system maintains one current inventory balance for each warehouse-product pair.

4.4.4 Supplier

SupplierID is the unique identifier of a supplier. It is the primary key of the Supplier entity. Its data type is **INT** and it must be required and unique.

SupplierName stores the supplier's name. Its data type is **VARCHAR(150)** and the value is required.

ContactName stores the name of the main contact person at the supplier. Its data type is **VARCHAR(100)**.

Phone stores the supplier's contact telephone number. Its data type is **VARCHAR(30)**.

Email stores the supplier's contact email address. Its data type is **VARCHAR(150)**.

Address stores the supplier's business address. Its data type is **VARCHAR(200)**.

4.4.5 SupplierProduct

SupplierID identifies the supplier that offers a product. It is a foreign key referencing **Supplier** and forms part of the composite primary key.

ProductID identifies the product offered by the supplier. It is a foreign key referencing **Product** and forms part of the composite primary key.

Together, **SupplierID** and **ProductID** uniquely identify a **SupplierProduct** record.

This entity represents the many-to-many relationship between suppliers and products and stores information specific to that supplier-product combination.

UnitPrice stores the supplier's current price for the product. Its data type is **DECIMAL(12,2)**. It represents the current supplier price and may change when the supplier provides a new price.

LeadTimeDays stores the expected number of days required for the supplier to deliver the product. Its data type is **INT**. It is used for delivery planning and supplier order management.

4.4.6 SupplierOrder

OrderID is the unique identifier of a supplier order. It is the primary key of the SupplierOrder entity. Its data type is **INT** and it must be required and unique.

OrderDate stores the date on which the supplier order is created. Its data type is **DATE** and it represents the beginning of the order lifecycle.

ExpectedDeliveryDate stores the expected date on which the order should arrive at the destination warehouse. Its data type is **DATE** and is used for delivery planning.

Status stores the current lifecycle state of the supplier order. Its data type is **VARCHAR(20)**. The order may begin as a draft and can only be submitted when it contains at least one order item.

SupplierID identifies the supplier receiving the order. It is a foreign key referencing **Supplier** and is required. Each **SupplierOrder** refers to exactly one supplier.

WarehouseID identifies the destination warehouse for the order. It is a foreign key referencing **Warehouse** and is required. Each **SupplierOrder** has exactly one destination warehouse.

4.4.7 SupplierOrderItem

OrderItemID is the unique identifier of an individual line within a supplier order. It is the primary key of the **SupplierOrderItem** entity. Its data type is **INT** and it must be required and unique.

OrderID identifies the supplier order to which the order item belongs. It is a foreign key referencing **SupplierOrder** and is required.

ProductID identifies the product being ordered. It is a foreign key referencing **Product** and is required.

Quantity stores the quantity of the product requested in the order line. Its data type is **DECIMAL(12,2)**. The quantity must represent a valid ordered amount.

UnitPrice stores the price agreed for the product at the time the order is created. Its data type is **DECIMAL(12,2)**. This is a historical price and does not automatically change when the supplier's current **UnitPrice** in **SupplierProduct** changes.

A product cannot appear more than once within the same **SupplierOrder**.

4.4.8 Shipment

ShipmentID is the unique identifier of a shipment. It is the primary key of the **Shipment** entity. Its data type is **INT** and it must be required and unique.

OrderID identifies the supplier order fulfilled by the shipment. It is a foreign key referencing **SupplierOrder** and is required. One supplier order may be fulfilled through multiple shipments.

TrackingNumber stores the external tracking reference assigned to the shipment. Its data type is **VARCHAR(50)** and is used to identify and track the physical delivery.

ShipmentDate stores the date on which the shipment is dispatched from the supplier or origin point. Its data type is **DATE**.

EstimatedArrival stores the current expected arrival date and time of the shipment. Its data type is **DATETIME** and is used for delivery planning.

ActualArrival stores the confirmed date and time when the shipment reaches the destination. Its data type is **DATETIME**. The value remains empty until arrival has been confirmed.

Status stores the current logistics state of the shipment. Its data type is **VARCHAR(20)**. The value is updated as the shipment progresses through its lifecycle.

4.4.9 ShipmentItem

ShipmentItemID is the unique identifier of a line within a shipment. It is the primary key of the ShipmentItem entity. Its data type is **INT** and it must be required and unique.

ShipmentID identifies the shipment carrying the allocated quantity. It is a foreign key referencing **Shipment** and is required.

OrderItemID identifies the supplier order item from which the shipment quantity is allocated. It is a foreign key referencing **SupplierOrderItem** and is required. The order item must belong to the same supplier order as the shipment.

Quantity stores the quantity of the order item allocated to this shipment. Its data type is **DECIMAL(12,2)**. The quantity must be positive, and the total quantity allocated across non-cancelled shipments cannot exceed the quantity originally ordered.

4.4.10 TrackingEvent

TrackingEventID is the unique identifier of a shipment tracking event. It is the primary key and has data type **INT**. It is required and uniquely distinguishes tracking events, including events that occur at the same time.

ShipmentID identifies the shipment associated with the tracking event. It is a foreign key referencing **Shipment** and is required.

EventTime stores the date and time at which the tracking event occurs. Its data type is **DATETIME** and it provides the chronological order of shipment events.

Location stores the geographical location where the tracking event is recorded. Its data type is **VARCHAR(150)**.

Status stores the shipment status reported by the tracking event. Its data type is **VARCHAR(30)**.

Description stores additional information about the event or handling activity. Its data type is **TEXT** and is optional.

4.4.11 ProductComponent

ParentProductID identifies the product that is assembled from one or more component products. It is a foreign key referencing **Product** and forms part of the composite primary key.

ComponentProductID identifies the product used directly as a component of the parent product. It is also a foreign key referencing **Product** and forms the second part of the composite primary key.

The combination of **ParentProductID** and **ComponentProductID** uniquely identifies a product-component relationship.

QuantityRequired stores the number of component units required to produce one unit of the parent product. Its data type is **DECIMAL(12,2)** and the value must be greater than zero.

ProductComponent represents a recursive relationship because Product can appear in two roles: as the parent product and as the component product. A product must not contain itself directly or indirectly.

4.4.12 StockMovement

MovementID is the unique identifier of an inventory transaction. It is the primary key of the StockMovement entity. Its data type is **INT** and it must be required and unique.

InventoryID identifies the inventory balance affected by the transaction. It is a foreign key referencing **Inventory** and is required.

ShipmentItemID identifies the shipment item that caused a receipt movement. It is an optional foreign key referencing **ShipmentItem**. This attribute is used only when MovementType is Receipt, and each ShipmentItem can be associated with at most one receipt movement.

MovementTime stores the date and time at which the inventory transaction is recorded. Its data type is **DATETIME** and provides the transaction timeline.

MovementType identifies the category of inventory change. Examples include **Opening, Receipt, Issue, and Adjustment**. Its data type is **VARCHAR(20)**. The movement type determines how the transaction should be interpreted.

QuantityDelta stores the signed change applied to inventory. Its data type is **DECIMAL(12,2)**. A positive value increases inventory, while a negative value decreases inventory. The resulting Inventory.Quantity must never become negative.

4.4.13 RestockAlert

AlertID is the unique identifier of a low-stock alert period. It is the primary key of the RestockAlert entity. Its data type is **INT** and it must be required and unique.

InventoryID identifies the inventory balance that caused the low-stock condition. It is a foreign key referencing **Inventory** and is required.

AlertDate stores the date and time when the low-stock condition is first detected. Its data type is **DATETIME**.

QuantityAtAlert stores the inventory quantity at the exact moment the alert is created. Its data type is **DECIMAL(12,2)**. This is a historical snapshot and should not change after the alert is created.

ReorderLevelAtAlert stores the reorder level at the moment the alert is created. Its data type is **DECIMAL(12,2)**. Like QuantityAtAlert, this is a historical snapshot.

ResolvedAt stores the date and time when the low-stock alert is resolved. Its data type is **DATETIME** and may be NULL. A NULL value means that the alert is still unresolved.

Message stores a human-readable explanation of the low-stock condition. Its data type is **VARCHAR(200)** and is used to provide additional information about the alert.

A low-stock condition occurs when **Inventory.Quantity < Inventory.ReorderLevel**. Each Inventory record may have at most one unresolved RestockAlert. When Quantity returns to or above ReorderLevel, the open alert is resolved; if Quantity later falls below the threshold again, a new alert period can begin.

4.5 Relationship Definitions, Cardinality and Participation

ID	Relationship	Cardinality	Participation / Meaning
R01	Warehouse–Inventory	1 : 0..N	Every Inventory belongs to exactly one Warehouse.
R02	Product–Inventory	1 : 0..N	Every Inventory belongs to exactly one Product.
R03	Supplier–SupplierProduct	1 : 0..N	Every SupplierProduct has one Supplier.
R04	Product–SupplierProduct	1 : 0..N	Every SupplierProduct has one Product.
R05	Supplier–SupplierOrder	1 : 0..N	Every order has exactly one Supplier.
R06	Warehouse–SupplierOrder	1 : 0..N	Every order has exactly one destination Warehouse.
R07	SupplierOrder–SupplierOrderItem	1 : 0..N	Draft may be empty; submitted order has at least one item.
R08	Product–SupplierOrderItem	1 : 0..N	Every order item selects one Product.
R09	SupplierOrder–Shipment	1 : 0..N	Every Shipment belongs to one order.
R10	Shipment–ShipmentItem	1 : 0..N	Draft may be empty; dispatched shipment has at least one item.
R11	SupplierOrderItem–ShipmentItem	1 : 0..N	Every shipment item is allocated from one order item.
R12	Shipment–TrackingEvent	1 : 0..N	Every tracking event belongs to one Shipment.
R13	Parent Product–ProductComponent	1 : 0..N	Every component link has one parent Product.
R14	Component Product–ProductComponent	1 : 0..N	Every component link has one component Product.
R15	Inventory–StockMovement	1 : 0..N	Every movement belongs to one inventory balance.
R16	Inventory–RestockAlert	1 : 0..N	Every alert belongs to one inventory balance.
R17	ShipmentItem–StockMovement	0..1 : 0..1	Only Receipt uses this link; a ShipmentItem is received at most once.

4.6 Function Definitions

ID	Function	Actor / Executor	Definition / Purpose	Related FR / BR
F-WH1	Register / update warehouse	Purchasing Staff	Create/edit warehouse identification and location/capacity information.	FR01
F-WH2	View warehouse inventory	Warehouse Staff;	View current Quantity and ReorderLevel for products at	FR07

		Inventory Manager	a warehouse.	
F-WH3	Confirm shipment receipt	Warehouse Staff	Confirm receipt of a ShipmentItem; creates Receipt StockMovement and updates Inventory.	FR08
F-WH4	Act as order destination	System	Set one destination Warehouse for an order; shipments inherit it.	FR04, BR06
F-PR1	Register / update product	Purchasing Staff	Manage product name, description, base unit and type.	FR02
F-PR2	Define product structure (BOM)	Authorized product-management role	Add/remove parent-component links with required quantity; system checks cycles.	FR09, FR10
F-PR3	View stock across warehouses	Inventory Manager	Check a product's Quantity across warehouses.	FR07
F-IN1	Maintain inventory balance	System	Maintain current Quantity and ReorderLevel for each warehouse-product pair.	FR07
F-IN2	Record stock movement	System	Create one StockMovement and update Quantity for each successful stock operation.	FR08
F-IN3	Evaluate restock condition	System	After Quantity/ReorderLevel changes, open or close a RestockAlert as required.	FR11
F-SU1	Register / update supplier	Purchasing Staff	Manage supplier contact information.	FR03
F-SU2	Manage product catalog & pricing	Purchasing Staff	Manage supplier-product offerings, current UnitPrice and LeadTimeDays.	FR03, BR02
F-SU3	Receive supplier orders	System / business process	Ensure an order has one supplier and that the supplier offers every ordered product.	FR04, BR05
F-SP1	Quote / update price	Purchasing Staff	Record the supplier's current UnitPrice for a product.	FR03
F-SP2	Record lead time	Purchasing Staff	Record expected LeadTimeDays for delivery planning.	FR03
F-SO1	Create order	Purchasing Staff	Open a draft order for one supplier and one destination warehouse.	FR04

F-SO2	Add / remove order items	Purchasing Staff	Add product lines and quantities; prevent duplicate product lines.	FR04, BR04
F-SO3	Submit order	Purchasing Staff	Change Draft to Submitted only when at least one item exists.	BR04
F-SO4	Track order status	Purchasing Staff; Inventory Manager	Monitor order status as shipments are created and received.	FR04
F-OI1	Record order line	System	Store product, ordered quantity and agreed historical price.	FR04, BR05
F-OI2	Check allocation limit	System	Ensure shipment allocations never exceed ordered quantity.	BR08
F-SH1	Create shipment	Logistics Staff	Create a shipment against a supplier order.	FR05
F-SH2	Allocate order item to shipment	Logistics Staff	Allocate order quantities across shipments within the ordered limit.	FR05, BR08
F-SH3	Dispatch shipment	Logistics Staff	Dispatch a shipment only after it has at least one ShipmentItem.	BR07
F-SH4	Confirm arrival	Warehouse Staff / Logistics Staff	Record ActualArrival and update shipment status when delivered.	FR06
F-SI1	Validate matching order	System	Reject a ShipmentItem linked to an order item from another order.	BR07
F-SI2	Trigger receipt	Warehouse Staff	Receive a ShipmentItem at most once and create one Receipt StockMovement.	FR08, BR13
F-TE1	Log tracking event	Logistics Staff	Append chronological status/location information to a shipment.	FR06
F-PC1	Add component link	Authorized product-management role	Define a parent-component pair and required quantity.	FR09
F-PC2	Validate cycle prevention	System	Reject direct or indirect BOM cycles.	FR10, BR11

F-PC3	Calculate multi-level requirement	System	Calculate total component requirements along valid product-structure paths.	Section 6
F-SM1	Log transaction	System	Create exactly one StockMovement for each successful opening, receipt, issue or adjustment.	FR08, BR12
F-SM2	Reject invalid issue	System	Reject an issue that would make Inventory.Quantity negative.	BR12
F-RA1	Create alert	System	Open an alert when Quantity < ReorderLevel and no alert is already open.	FR11, BR14
F-RA2	Close alert	System	Set ResolvedAt when Quantity recovers to ReorderLevel or above.	FR11, BR15
F-RA3	Prevent duplicate alerts	System	Keep the existing open alert while the low-stock condition persists.	BR14

4.7 User Permission Definitions

Actor / Role	Allowed Functions	Permission Scope
Purchasing Staff	F-WH1, F-PR1, F-SU1, F-SU2, F-SP1, F-SP2, F-SO1, F-SO2, F-SO3, F-SO4	Supplier, product, pricing and purchasing operations.
Warehouse Staff	F-WH2, F-WH3, F-SI2, inventory issue/adjustment operations	Warehouse receipt and inventory-side operations.
Logistics Staff	F-SH1, F-SH2, F-SH3, F-SH4, F-TE1	Shipment creation, allocation, dispatch, arrival and tracking.
Inventory Manager	F-WH2, F-PR3, F-SO4; review RestockAlert	Inventory monitoring and replenishment decisions.
System / automatic processing	F-WH4, F-IN1, F-IN2, F-IN3, F-OI2, F-SI1, F-PC2, F-PC3, F-SM1, F-SM2, F-RA1, F-RA2, F-RA3	Automatic validation, balance update, BOM checks and alert logic.

4.8 Key Business Constraints

Constraint	Definition	Rule
Inventory uniqueness	One Warehouse–Product pair has at most one Inventory record.	BR01
Supplier-product validity	The order supplier must offer every ordered product.	BR05
Order uniqueness	A Product cannot repeat in the same SupplierOrder.	BR04
Partial shipment	Non-cancelled shipment allocations cannot exceed	BR08

limit	ordered quantity.	
Shipment-order consistency	ShipmentItem and Shipment must belong to the same SupplierOrder.	BR07
Inventory non-negativity	Quantity and ReorderLevel cannot be negative; invalid issues are rejected.	BR12
Receipt uniqueness	A ShipmentItem may be received at most once.	BR13
Recursive structure validity	No direct/indirect Product cycle; QuantityRequired > 0.	BR10–BR11
Restock alert uniqueness	At most one unresolved RestockAlert per Inventory.	BR14
Alert lifecycle	Low stock opens an alert; recovery closes it; a later low-stock period opens a new one.	BR14–BR15

4.9 Important Modelling Clarifications

Entity vs Actor: Warehouse, Product, Inventory, Supplier and the other listed objects are database entities. Purchasing Staff, Warehouse Staff, Logistics Staff and Inventory Manager are actors/roles.

Current state vs history: Inventory stores the current operational balance, while StockMovement stores the history of successful inventory changes and RestockAlert stores alert periods.

Current price vs historical price: SupplierProduct.UnitPrice is the current supplier quotation; SupplierOrderItem.UnitPrice is the agreed price captured for that order.

User action vs system action: A user initiates a valid business action; the system then performs required automatic updates and validations.

Recursive roles: ParentProductID and ComponentProductID refer to the same Product entity but represent different roles.

Phase 1 vs Phase 2: Phase 1 defines the conceptual model and business constraints. DBMS-specific SQL constraints, triggers, indexes and implementation details are deferred to Phase 2.

V. Extended Entity Relationship Model

The diagram below is the complete Phase 1 EER model. The symbol 1 means exactly one record, and 0..N means zero to many records. Relationships R01 through R17 are defined precisely in Section 4.

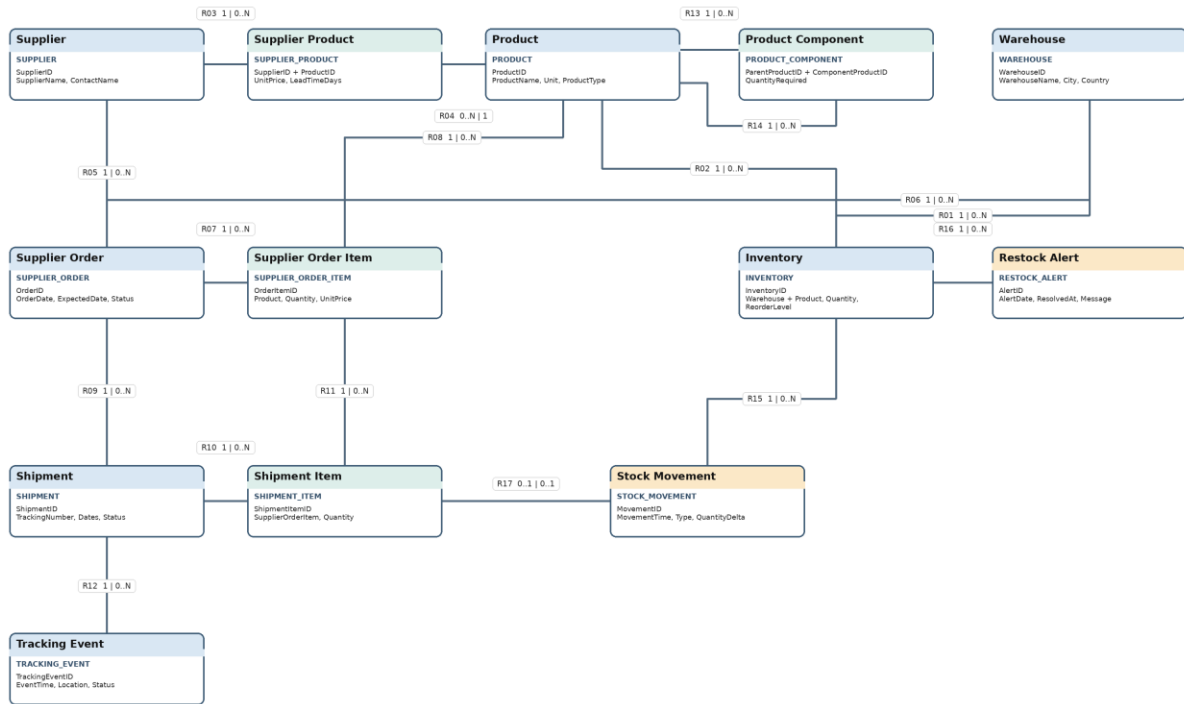


Figure 1 Complete EER model for warehouse logistics product structures and restock alerts3

VI. Recursive Relationships and Inventory Alerts

Recursive Product Structure

Product participates in two roles within the same structure. The first role is the parent product that must be assembled. The second role is the component product that is consumed. ProductComponent is the associative entity that records a direct edge between these roles and stores QuantityRequired.



Figure 2 ProductComponent represents two Product roles in the recursive relationship

For example, KIT-A contains 2 MODULE-B units, and each MODULE-B contains 4 SCREW-C units. One KIT-A therefore requires $2 \times 4 = 8$ SCREW-C units along this path. If SCREW-C also appears on another valid branch, the system adds the requirements from those paths.

Before adding a Parent to Component link, the system must reject equal product identifiers and recursively search from the proposed Component through its existing descendants. If the proposed Parent is found, the new link would create a cycle and must be rejected. This rule applies to both inserts and updates.

Restock Alert Logic

Condition After Inventory Change	Required Action
Quantity < ReorderLevel and no open alert exists	Create a new RestockAlert, capture the quantity and threshold, and leave ResolvedAt null.
Quantity < ReorderLevel and an open alert exists	Keep the existing alert and do not create a duplicate record.
Quantity >= ReorderLevel and an open alert exists	Set ResolvedAt to the transaction time to close the alert.
Quantity >= ReorderLevel and no open alert exists	Take no alert action.



Figure 3 Inventory update and restock alert evaluation flow

The alert condition is evaluated when Inventory is created, Quantity changes, or ReorderLevel changes. Updates to StockMovement, Inventory, and RestockAlert must all commit or all roll back. The database must maintain at most one unresolved alert for each Inventory record even when concurrent transactions occur.

Phase 1 specifies the required trigger behavior. The SQL trigger, row-locking method, and concurrency tests will be implemented after the group selects a database management system.

VII. Validation Cases and Completion Status

Acceptance Cases

Case	Data or Action	Expected Result
Unique inventory	Add a second record for the same Warehouse and Product.	Reject the duplicate pair.
Multi-level structure	KIT to MODULE = 2 and MODULE to SCREW = 4.	Calculate 8 SCREW units for one KIT along this path.
Cycle prevention	After the structure above, add SCREW to KIT.	Reject the link because it creates an indirect cycle.
Partial delivery	Order 100 units and allocate 40 to SH1 and 60 to SH2.	Accept both; reject one additional unit.
Wrong order	A ShipmentItem refers to an item from another order.	Reject it even though the individual records exist.
No duplicate alert	Threshold 50; quantity changes 60 to 20 to 15.	Create one alert at 20 and no second alert at 15.
Close and reopen	Quantity changes 15 to 50 to 49.	Close the alert at 50 and create a new period at 49.
Duplicate receipt	Submit the same ShipmentItem for receipt twice.	Only the first receipt may change inventory.

Phase 1 Requirements Checklist

Required Content	Status	Location
Problem description and objectives	Complete	Section 1
Scope users and assumptions	Complete	Section 1
Functional requirements	Complete	Section 2
Business rules	Complete	Section 3
Entities attributes and identifiers	Complete	Section 4
Relationship cardinality and participation	Complete	Section 4
Complete EER model	Complete	Section 5
Multi-level recursive relationship	Complete	Section 6
Restock alert trigger logic	Complete	Section 6